

Basic of Switching

→ Every physical device's NIC card contains a MAC Address

→ Wifi adapter, Ethernet port in laptops are separate, different MAC addresses.

→ In general, Ethernet level Frame transfer never happens at routers, happens within individual LANs only.

→ Every ethernet frame contains

- Source MAC Address
- Destination MAC address
- upper layer data.

→ If A, B exist in same LAN and A wants to send B a frame.

A has to create a frame put A's mac at source, B's mac addr at destination, and send.

we'll see how that works.

"LAN is a broadcast domain" Full duplex
Switched ethernet

↓
Means any broadcast message
Should reach entire LAN,
all devices in LAN and
nothing outside.

⇓
So Every Switch should have
a path to the Core switch
and the uplink router,
& to every other device in the
LAN.

a PC is connected to
a single port of a
Switch. and PC → Switch,
Switch → PC has separate
channels.

→ if both PC, Switch send
to each other → NO
collisions
will happen.

Consider a simple LAN.

main/core switch
of LAN

up link to router.



PC1 PC2 PC3 PC4

M_1 M_2 M_3 M_4

MAC
addresses →

Every switch has a table called MAC
address table → it helps in guiding every
frame to correct destination MAC

What does a MAC address table contain (in prev example only for now)

So, in prev example, MAC table should have

Port 1 — M1 mac address

Port 2 — M2

Port 3 — M3

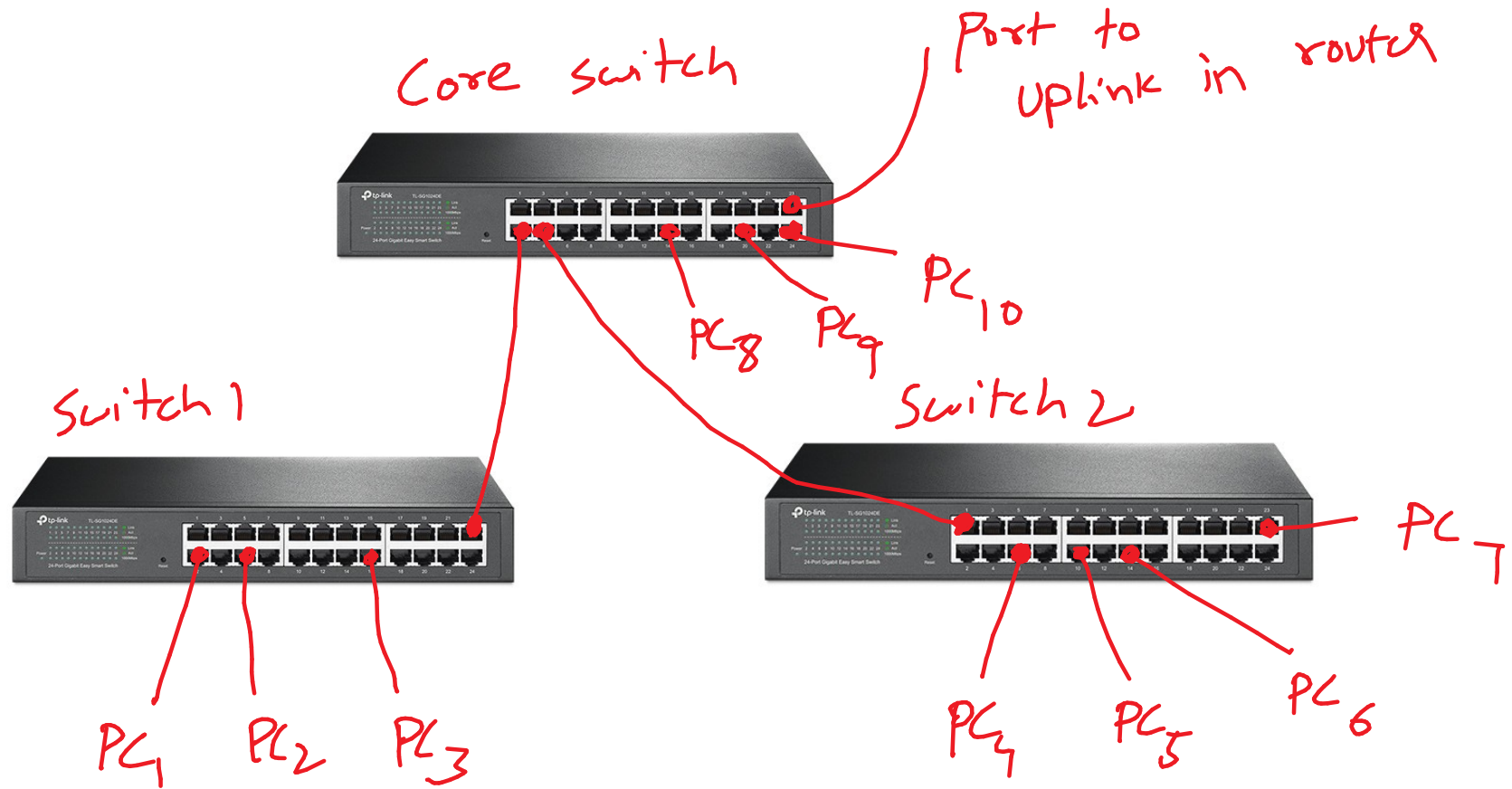
Port 8 — M4

Port 24 — Uplink router port's MAC address

↓

the port in the switch to which whole LAN is connected. The gateway to whole LAN

But Switches usually are multiple, like



Switch's MAC address no one cares but.

Individual switches connected to PCs directly store which port and which MAC address.

But we also need to make sure PCs connected to different switches can also communicate.

→ So for ports that are connected to another switch, we store

< port no, < list of MAC addresses
of all reachable devices
From this port >

Core switch

Port to uplink in router

Switch 1

Switch 2



In this case,
MAC tables
look like this,

Core switch

Port 1 - PC₁, PC₂, PC₃

Port 2 - PC₄, 5, 6, 7

P₇ - PC₈, ---

Switch 1

Port 1	PC ₁ MAC
Port 3	PC ₂ MAC
Port 8	PC ₃ MAC
Port 24	PC ₄ , 5, 6, 7, 8, 9, 10, Uplink port

Switch 2

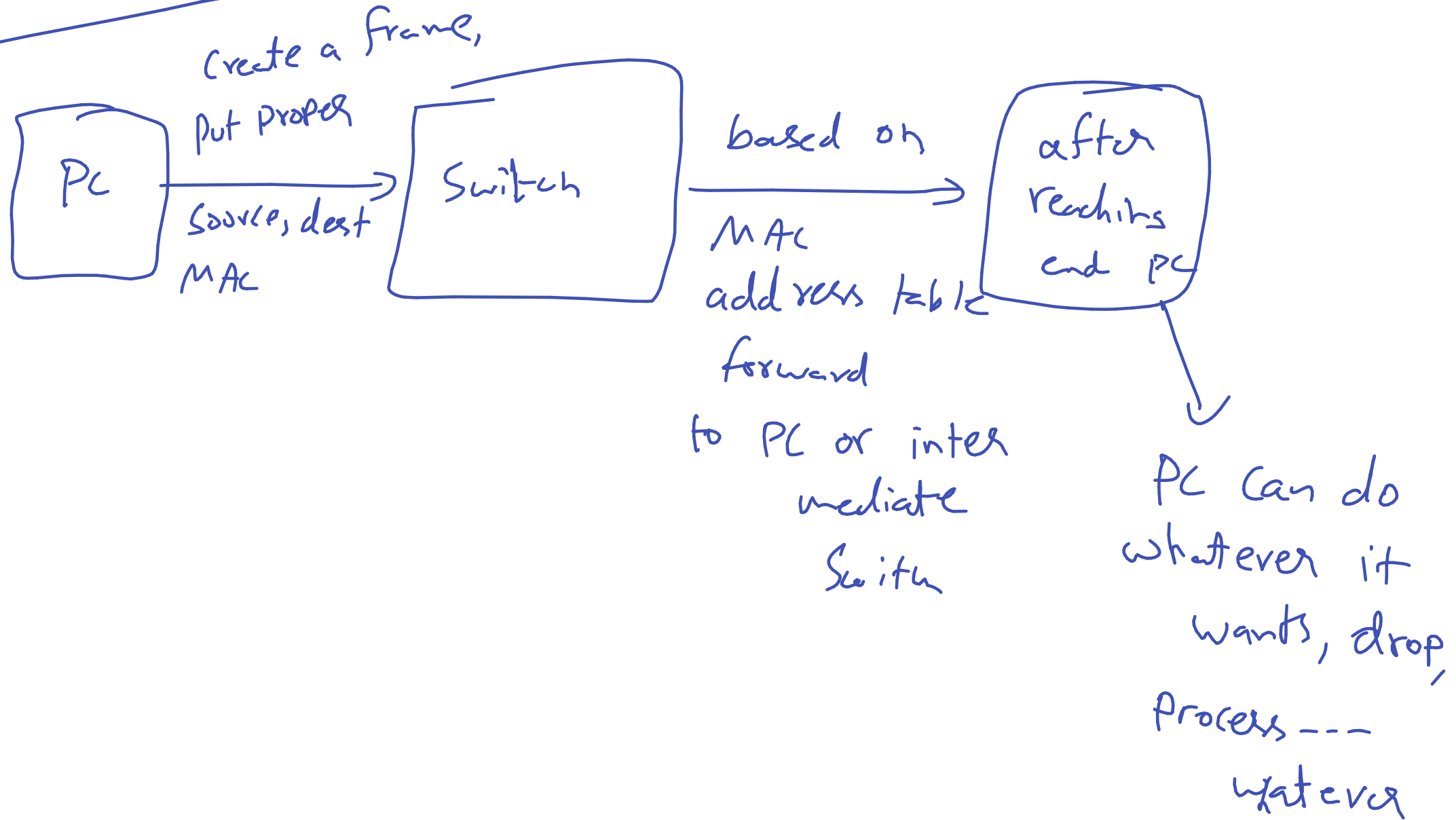
Port 13 - PC₁, 2, 3, 8, 9, 10, Uplink port

Port 3 - PC₄, Port 5 - PC₅, P₇ - PC₆,

P₂₇ - PC₇

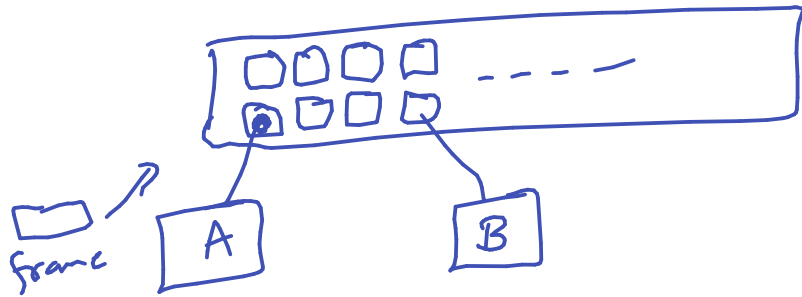
This is the end, what we want to achieve.

Switching



Now we have MAC address tables,

if A wants to send B a frame, it creates a frame with
src as MAC_A , Dest as MAC_B .



→ A sends frame to switch.

→ Switch looks which port can reach this MAC of B, sends

→ If no port exists, switch broadcasts, only B keeps, rest all drops.

Some questions we missed

① How is the table built?

the table always changes $\rightarrow$ if new pc is connected to switch and it sends a frame, then switch learns

Port $i \rightarrow PC_j$

② How to know dest MAC address

Usually from IP layer, we mostly want to interact with outside the LAN sites and services $\rightarrow$ for that

we go through our gateway

gateway device.

→ So every device must know destination MAC address of gateway device. (we only know its gateway IP)

→ Also if we want to interact with a IP address that's inside our subnet → we need to know its MAC address

↓
So How to know destination MAC address, we only know IP of them?

(if you want to talk through gateway is cool also but no one does that

ARP Protocol

(literally no one)

ARP (address resolution Protocol)

→ Every system in LAN runs an ARP software.

→ if a client wants to know some one's Mac,

he sends/broadcasts "ARP: Who owns this ip '172.---'"

→ the concerned person replies,

"ARP: i own the ip '172.---' and this is my
MAC: 'E8:4 ----'"

and now source knows destination mac, they chat.

Even every device knows the gateway's mac like this only

STP to prevent a sending in LAN for
broadcasting.

IPv4

till now we saw Switching in LAN only

this is the above layer,

IPv4 address and Subnetting.

4 bytes — 32 bits — Same capacity of
an int variable.

Subnet → a subset of IP addresses.

a Subnet looks like this.

represents how many

IP addresses are
there in this subnet

172.168.0.0 / 26
↓
network address

Subnet size

write binary representation.

10101100.10101000.00000000.00000000

this Subnet represents set of

IP addresses whose first 26 bits

are exactly the same as above

and remaining 6 bits can be

anything. from 000000 to 111111 →

we can make $2^6 = 64$

IP addresses in this subnet

We can represent whole ip address space as

$$0.0.0.0/0 \approx /00000000.00000000.00000000.00000000$$

all bits we can change.

if we want to keep 172, 168 only constant in 172.168.0.0

we can write

$$172.168.0.0/16 \quad (\text{or}) \quad 172.168.0.1/16 \quad \text{or} \dots$$

↙ any bit after the '/' can be anything.

it contains $2^{32-16} = 2^{16}$ different IP addresses.

Generally, we allocate one subnet per LAN

Giving LANs individual subnets makes routing easier, efficient.
Small routing tables.

How to split subnets

Given a $172.168.48.0/20$ subnet with $2^{32-20} = 2^{12}$ devices,
we can split it into 2 2^{11} size subnets.

$$2^{12} = 2^{11} + 2^{11} \text{ very easily.}$$

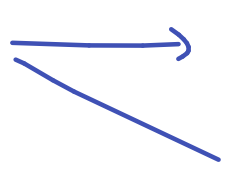
$172.168.48.0/20$ is

$10101100.10101000.00110000.00000000$

We can split this into

$172.168.00110/000.00000000$

$172.168.00111/000.00000000$

in the 12 bits we have with freedom, we made it into
2 subnets,  1st bit fix 0, remaining freedom bits
1st bit fix 1, remaining freedom bits.

both are now individual 2nd subnets we can write as

172.168.48.0/21, 172.168.56.0/21

this way we can split subnets easily.

Exercise

4. what is the size of the smallest possible subnet that can give separate subnets for 4 divisions containing 60, 62, 12, 120 devices respectively

Note: In a IPv₄ Subnet we have 2^k ip addresses but we are not allowed to set the k freedom bits to $\underbrace{000 \dots 0}_k$ and $\underbrace{111 \dots 1}_k$. So for devices we can only give $2^k - 2$ ip addresses only.

for 60 $\rightarrow$ we need a $62 \approx 64$ size subnet

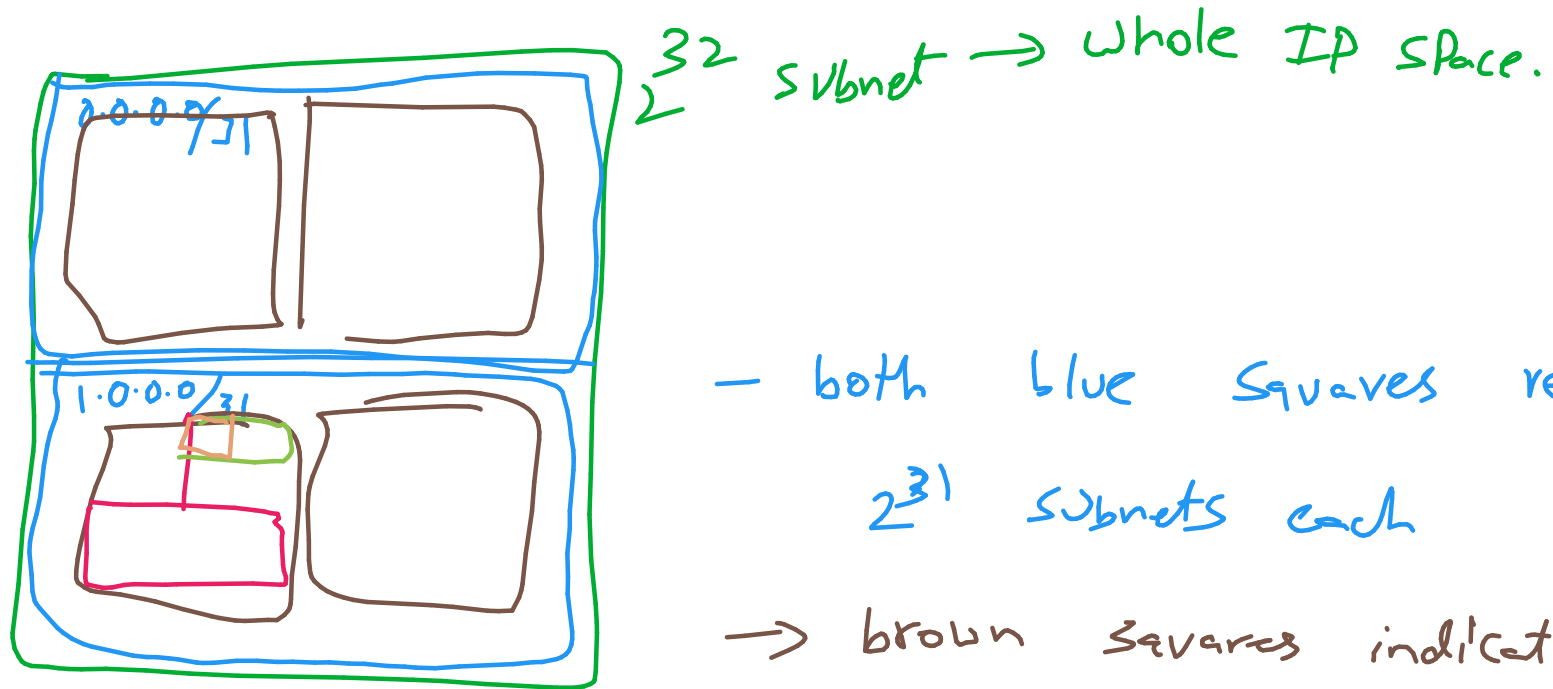
62 $\rightarrow$ $62 + 2 = 64 \approx 64$ size subnet

12 $\rightarrow$ $12 + 2 = 14 \approx 16$

120 $\rightarrow$ $122 \approx 128$ size subnet.

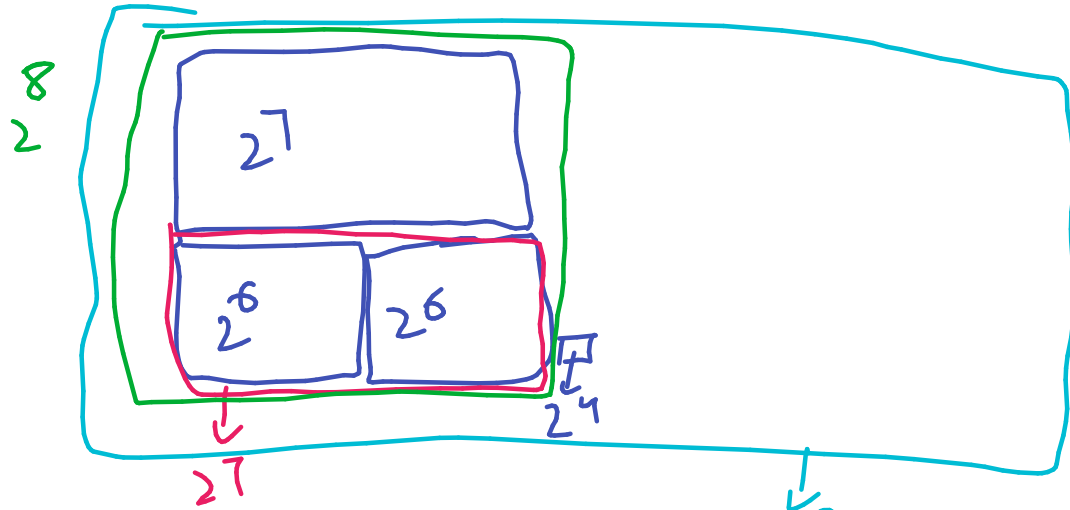
The question is, what is the smallest subnet which can be carefully sliced to allocate individual subnets for this.

Imagine dividing subnets like dividing a square.



--- $\rightarrow$ that's how subnets are like.

Coming to our question. Find smallest possible square that can fit a 2^6 , 2^6 , 2^4 , 2^7 shapes.



$2^9 \rightarrow$ So we need a 2^9 subnet at least to allocate. $\rightarrow$ we can do easily with bigger size also but we shouldn't waste the extra spaces.

there is no other way to fit 2^4 square to make whole thing small.

$\rightarrow$ if 2^4 didn't exist, we could've done using 2^8 only directly.

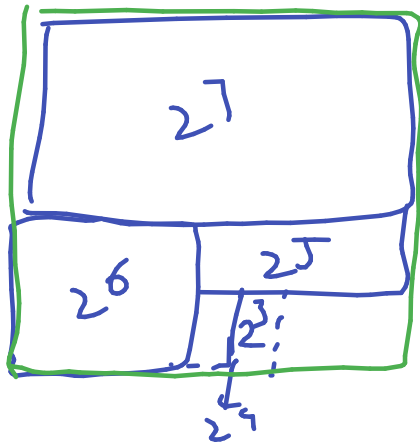
Ex 2 :- Smallest possible subnet that can allocate.

Seperate subnets for 100 devices, 50, 20 devices, 5 devices
separately.

$$100 \rightarrow 2^7 \quad 20 \rightarrow 2^5$$

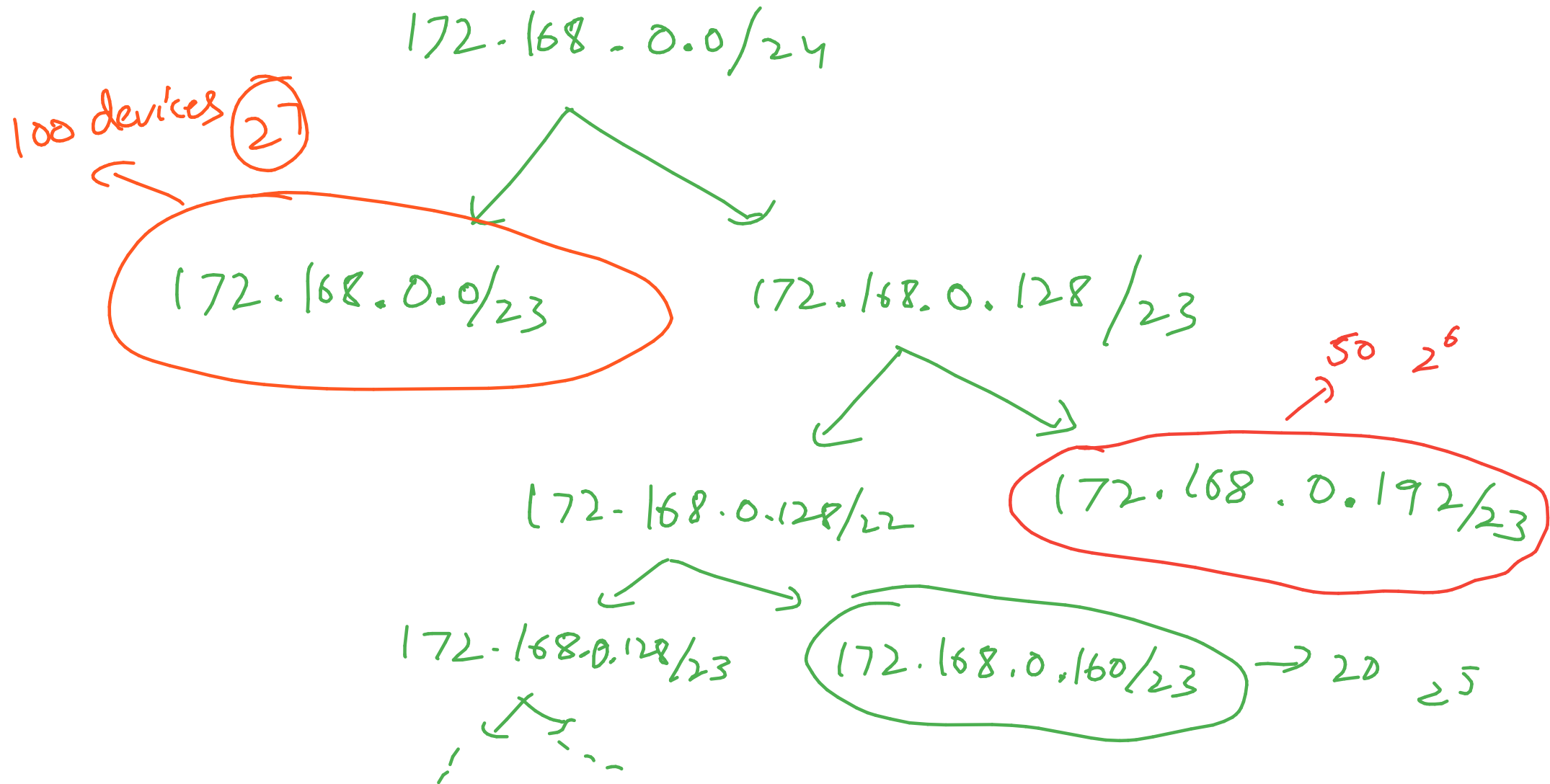
$$50 \rightarrow 2^6 \quad 5 \rightarrow 2^3$$

Just try to fit tight in a
square,



a 2^8 Subnet can be enough.

ex-3 :- Given a $172.168.0.0/24 \rightarrow$ in prev question, write what subnets you can assign to each subset.



Given a Subnet and a Particular IP address, How to determine whether that IP lies inside Subnet Subset.

Ans:- Just check the network bits are Same or not, ignore freedom bits.

ex:- 172.168.138.128/26 Subnet, 172.168.138.160

and

172.168.138.100

write individual bit patterns.

10101100 . 10101000 . 10001010 . 10000000

and

10101100 . 10101000 . 10001010 . 10100000 → 172.168.138.160 ✓ belongs in Subnet

10101100 . 10101000 . 10001010 . 01100100 → 172.168.138.100 X not belongs.

Only need to check the network bits same (or) not.

Subnet mask we write it as /24, /26 ...
we can also write like

11111111.11111111.11111111.00000000 $\rightarrow$ /24

11111111.11111111.11111111.11000000 $\rightarrow$ /26

To check IP addr belongs in subnet,

we can also do

$$\boxed{(\text{IP addr}) \&\& (\text{Subnet mask})} == (\text{network address})$$

$\downarrow$
if this returns true, then

yes. IP belongs in subnet.

$\rightarrow$ from ^{one} IP, Subnet mask we can determine
network mask using the above one.

